SIAM J. DISC. MATH.
Vol. 2, No. 3, pp. 307–321, August 1989

004

PERFORMANCE GUARANTEES ON A SWEEP-LINE HEURISTIC FOR COVERING RECTILINEAR POLYGONS WITH RECTANGLES*

D. S. FRANZBLAU†

Abstract. Finding the minimum number of rectangles required to cover a rectilinear or orthogonal polygon, where overlapping of rectangles is allowed, is one of several well-known, hard geometric decomposition problems. This paper reports the first results known that give worst-case performance bounds for an approximation algorithm for this problem. It is proved that partitioning the polygon into rectangles (with no overlapping) produces at most $2\theta + h - 1$ rectangles, where θ is the minimum number of rectangles in a cover, and h is the number of holes. Examples are also given in which this bound is tight. The proof is based on counting arguments, and on Euler's formula for a planar graph. This paper shows that extending rectangles vertically and deleting duplicates produces at most $O(\theta \log \theta)$ rectangles. For the proof, geometric constraints are used to construct a large antirectangle, a set of points with no two contained in the same rectangle. This algorithm has an $O(n \log n)$ implementation using balanced trees, where n is the number of vertices.

Key words. polygon, cover, rectangle, rectilinear, orthogonal, algorithm, approximation, heuristic, sweep-line

AMS(MOS) subject classifications. 05B40, 05B50, 52A45, 68Q25, 68R05

1. Introduction.

The problem of covering a rectilinear or orthogonal polygon with the minimum number of rectangles is one of a large class of geometric decomposition problems (several of which are surveyed in O'Rourke and Supowit (1983)). There has been little progress in finding algorithms for constructing minimum covers of arbitrary polygons with primitive shapes, thus, much attention has been focused on rectilinear geometry. The problem has applications to storing graphical images (Masek (1978)) and to the manufacture of integrated circuits (Mead and Conway (1980), Chaiken et al. (1981), Hegedüs (1982)). This and related problems have given rise to new classes of perfect graphs of interest in combinatorics (Saks (1982), Motwani, Raghunathan, and Saran (1988a), (1988b)).

The exact computational complexity of even this seemingly simple problem has remained open for a number of years. Masek (1978) proved that the problem is NP-hard for arbitrary polygons. Conn and O'Rourke (1987) showed that several restricted versions of the problem are also NP-hard. On the other hand, Franzblau and Kleitman (1984) gave a polynomial-time algorithm for covering vertically convex polygons with rectangles. They used a combinatorial duality theorem of Györi (1984), which improved an earlier result by Chaiken et al. (1981). Lubiw (1985), (1988a) showed that the problem was polynomial time for a somewhat larger class of polygons and has reported a weighted version of Györi's theorem (Lubiw (1988b)). Until now, the complexity of the problem for polygons with no holes was unknown. However, in a recent abstract, Culberson and Reckhow (1988) have reported that the problem is NP-hard even in this case.

The problem of covering a rectilinear polygon with the minimum number of "orthogonally convex" or "orthogonally star-shaped" components has a similar history (Keil (1986), Reckhow and Culberson (1987), Reckhow (1987), Culberson and Reckhow (1988), Motwani, Raghunathan, and Saran (1988a), (1988b)).

We distinguish the problem of *covering*, where overlapping of rectangles is allowed, from that of *partitioning* into disjoint rectangles. By contrast, the problem of partitioning an arbitrary rectilinear polygon into the minimum number of rectangles has an

* Received by the editors August 22, 1988; accepted for publication December 15, 1988.
† Department of Mathematics, Vassar College, Poughkeepsie, New York 12601.

$O(n^{5/2})$ solution (Ohtsuki (1982), Pagli et al. (1979)). When degenerate point holes are allowed, however, the problem is NP-hard (Lingas (1982)).

Since good algorithms for minimum cover problems are so scarce, it is worthwhile to look for approximation algorithms with guaranteed performance bounds. (See Garey and Johnson (1979, chap. 6) for an introduction to this approach.) This paper provides what we believe are the first results in this direction. Other covering heuristics have been reported, but with no analysis of performance bounds (Hegedüs (1982), Levcopoulos (1985)).

We first show that partitioning the polygon into rectangles using horizontal cuts produces at most twice the minimum number of rectangles if the polygon has no holes. More generally, partitioning always produces at most $2\theta + h - 1$ rectangles, where θ is the minimum number of rectangles in a cover, and h is the number of holes in the polygon. We also give examples in which this bound is tight.

We then study a simple covering heuristic, called *Partition/Extend*, in which the rectangles in the partition are stretched vertically as far as possible, and duplicates are deleted. We show that this heuristic produces at most $O(\alpha \log \alpha)$ rectangles, where α is the size of an antirectangle, a set of points such that no two can be contained in the same rectangle. This gives a bound of $O(\theta \log \theta)$, since $\alpha \leqq \theta$. In our examples, the number of rectangles produced is at most 3θ, which we conjecture is the correct upper bound.

The organization of the paper is as follows. Section 2 contains basic definitions. The covering heuristic and main results are stated formally in § 3. An $O(n \log n)$ implementation of the heuristic is described in § 4. Some observations and open problems are given in § 5.

2. Definitions. A *rectilinear* or *orthogonal polygon* is a finite set of unit squares (or "pixels") on a two-dimensional, integer grid. That is, it is a simple polygon with integer vertices, edges aligned with the horizontal and vertical axes, and positive area. Such a polygon is also called a *polyomino*. In applications, all polygons are finite, so there is no loss of generality in assuming the polygon is embedded in a grid.

A *rectangle* will denote a rectangle (in the usual sense) which is a union of unit squares. That is, it is a rectangle aligned with the horizontal and vertical axes.

Given a rectilinear polygon R, a *rectangle cover* is a finite set of rectangles whose union is equal to R. Rectangles are allowed to overlap, but must be contained completely within R. A *minimum rectangle cover* is a rectangle cover containing the minimum possible number of rectangles. We denote the number of rectangles in a minimum cover by $\theta(R)$.

Dual to a rectangle cover is an *antirectangle* or *independent set*, a subset A of unit squares in R, such that every rectangle inside R contains at most one square of A. We denote the maximum number of squares in an antirectangle by $\alpha(R)$. It is easy to see that $\alpha(R) \leqq \theta(R)$. (See Fig. 1.) (Note that there are examples in which $\alpha < \theta$. See Chaiken et al. (1981), or Franzblau and Kleitman (1984, p. 166).)

Regarding R as a closed planar region, the boundary of R consists of horizontal and vertical line segments called *edges*. Edges intersect only at endpoints. A *vertex* of R is the intersection of a horizontal and a vertical edge. There are three types of vertices, shown in Fig. 2. A *normal convex vertex* is the intersection of exactly two edges which form a 90° angle inside R. A *degenerate convex vertex* is the intersection of two pairs of edges forming two 90° angles. A *concave vertex* is the intersection of two edges forming a 270° angle.

We regard the vertices and edges of R as a straight-line planar graph. A *hole* in R is a bounded (finite area) face of this graph, contained in the complement of R. R is

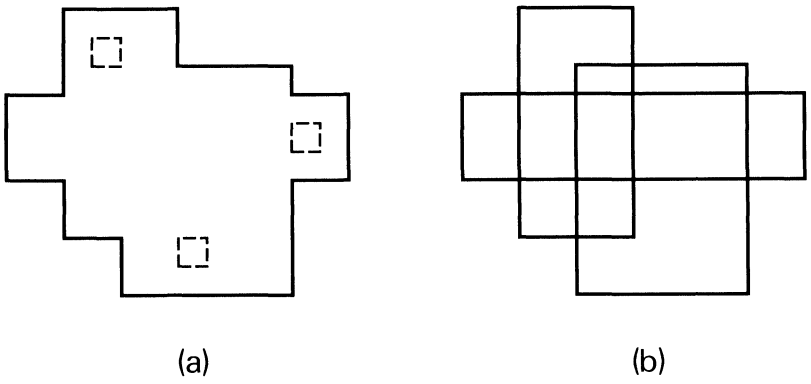

FIG. 1. (a) *Rectilinear polygon with antirectangle of size three (squares with dashed outlines).* (b) *Minimum rectangle cover, also of size three.*

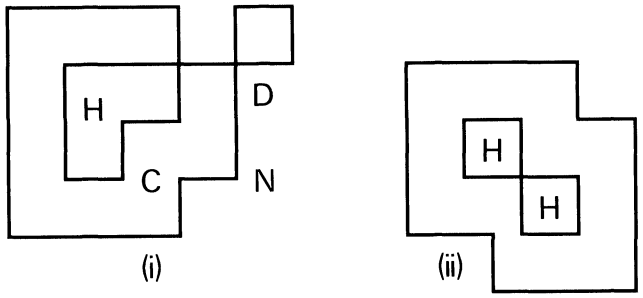

FIG. 2. *Polygon with two connected components,* (i) *and* (ii), *and three holes (H). Vertex N is normal convex, D is degenerate convex, and C is concave.*

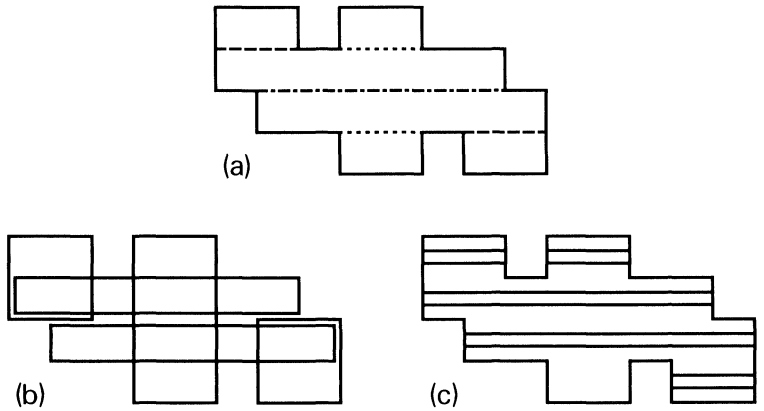

FIG. 3. (a) *Horizontal partition. Two type* 1 *chords* (– – –), *one type* 2 *chord* (–·–·), *and two type* 3 *chords* (· · ·) *are shown.* (b) *Rectangle cover determined by extending rectangles vertically. (Note that the two long horizontal rectangles are shown smaller than actual size for clarity.)* (c) *Pairs of parallel lines represent five inequivalent horizontal slices, which determine the same rectangle cover.*

connected if, after deleting the holes of R (along with their boundaries), the remaining boundary is a connected graph. (See Fig. 2.)

A *horizontal chord* of R is a horizontal line segment contained in R, such that at least one endpoint is a concave vertex, both endpoints lie on the boundary, and no other point lies on the boundary. (See Fig. 3(a).) Every concave vertex determines a unique horizontal chord. This notation is taken from Ohtsuki (1982).

3. The covering heuristic and worst-case performance guarantees. The covering heuristic can be described as follows.

ALGORITHM *Partition/Extend*.

Input: Rectilinear polygon R.
Output: Rectangle cover for R.

1. *Partition* R into rectangles by cutting along each horizontal chord of R. (See Fig. 3(a).)
2. *Extend* each rectangle vertically inside R until it is vertically maximal, i.e., touches the boundary on top and bottom. Delete any repeated rectangles. (See Fig. 3(b).)

Let R be a rectilinear polygon, and let $p = p(R)$ be the number of rectangles obtained by horizontal partitioning alone (step 1). Let $\bar{p} = \bar{p}(R)$ be the number of rectangles obtained by Algorithm *Partition/Extend*.

Recall that $\theta = \theta(R)$ is the minimum number of rectangles in a cover, and $\alpha = \alpha(R)$ is the maximum number of unit squares in an antirectangle or independent set. Let $h = h(R)$ be the number of holes of R.

The main theorems of this paper are as follows.

THEOREM 1. *For any connected rectilinear polygon*, $p \leqq 2\theta + h - 1$.

THEOREM 2. *For any rectilinear polygon*, $\bar{p} \leqq 2\alpha(\log \alpha + 1)$.

From these theorems and the inequalities $\bar{p} \leqq p$ and $\alpha \leqq \theta$, we immediately derive the following performance bounds for Algorithm *Partition/Extend*.

COROLLARY 1. (Theorem 1.) *If R has no holes, then* $\bar{p}/\theta \leqq 2$.

COROLLARY 2. (Theorem 2.) *For any rectilinear polygon*, $\bar{p}/\theta \leqq O(\log \theta)$.

Figures 4 and 5 give examples in which $p = 2\theta + h - 1$, so the bound of Theorem 1 is tight. In Fig. 4(a), $\bar{p} = p$, so the bound of Corollary 1 is tight.

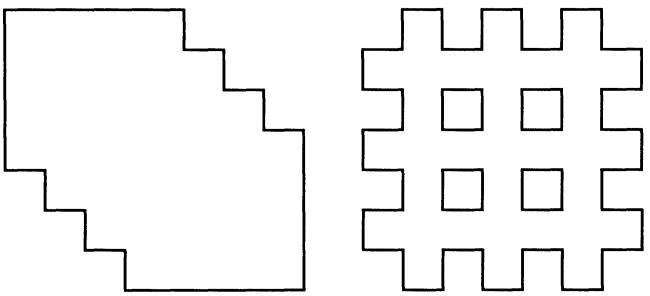

FIG. 4. *Two examples in which Theorem 1 is tight*: $p = 2\theta + h - 1$. (a) *Chain of θ rectangles* ($\theta = 4$). *There are no holes, and* $p = 2\theta - 1$. (b) *Lattice with $\theta = 2k$ rectangles* ($k = 3$). *The number of holes is* $(k-1)^2$, *and* $p = 2k + k^2 = 2\theta + h - 1$.

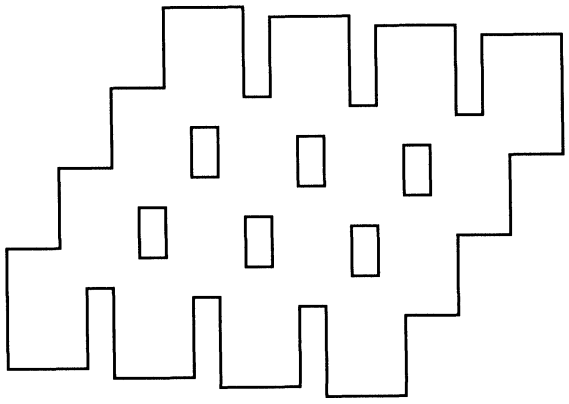

FIG. 5. *Another example in which Theorem* 1 *is tight. Formed from k rows of staircases of k non-overlapping rectangles each* ($k = 4$, $\theta = k^2$, $h = (k-1)(k-2)$). *The top staircase adds k rectangles to the partition. Every subsequent staircase adds* $3k - 1$ *rectangles. So* $p = k + (3k-1)(k-1) = 2\theta + h - 1 \leqq 3\theta$. *Extending rectangles vertically does not reduce the number.*

Figure 4(b) shows that the number of holes can be $\Theta(\theta^2)$, making the bound of Theorem 1 unsatisfactory in general. Note, however, that the heuristic gives the optimal solution in this case.

To prove Theorem 1, we find a relation between the minimum number of rectangles in a cover and the number of horizontal chords and vertices. We then relate the number of chords to the number of holes using Euler's formula for a planar graph.

There are three types of chords, illustrated in Figure 3(a). A *type* 1 *chord* contains exactly one concave vertex, a *type* 2 *chord* connects two concave vertices with opposite orientations, and a *type* 3 *chord* connects two concave vertices with the same orientation. Let μ_1 and μ_2 be the number of normal convex vertices and degenerate convex vertices, respectively. Let c_1, c_2, and c_3 be the number of type 1, type 2, and type 3 horizontal chords. Recall that θ is the number of rectangles in a minimum cover. We then have the following useful relation.

LEMMA 1.

$$(*) \qquad \mu_1 + 2\mu_2 + c_1 + 2c_2 \leqq 4\theta.$$

Proof. 4θ is the number of corners of rectangles in a minimum cover. Each normal convex vertex of R is also the corner of at least one covering rectangle. Each degenerate convex vertex yields two distinct corners. Given a type 1 chord, with concave vertex u as an endpoint, there must be a rectangle covering u whose corner lies on the chord. Using the same reasoning, each type 2 chord contains at least two rectangle corners. (See Fig. 6.) Since we have not counted any corners twice, the number of corners is at least $\mu_1 + 2\mu_2 + c_1 + 2c_2$. □

We can prove a similar relation involving p, the number of rectangles in the horizontal partition of R.

LEMMA 2.

$$(**) \qquad \mu_1 + 2\mu_2 + 3c_1 + 2c_2 + 2c_3 = 4p.$$

Proof. $4p$ is now the number of corners of rectangles in the *partition*. Each normal convex vertex is also a rectangle corner; each degenerate convex vertex yields two rectangle corners. Each type 1 chord determines three corners, while each type 2 or type 3 chord determines two corners. (See Fig. 7.) □

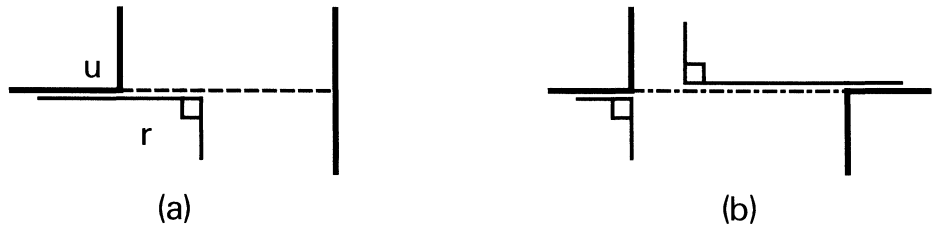

FIG. 6. *Illustration for the proof of Lemma* 1. (a) *Vertex u is the concave endpoint of a type* 1 *chord. There must exist a rectangle r covering u, with a corner on the chord.* (*Rectangle r is offset slightly from its actual position.*) (b) *Endpoints of a type* 2 *chord must be covered by two distinct rectangles with corners on the chord.* (*The two rectangles are offset slightly.*)

THEOREM 1. *For any connected rectilinear polygon,* $p \leqq 2\theta + h - 1$.

Proof. From (∗) and (∗∗) we obtain

$$(***) \qquad 2p \leqq 2\theta + (c_1 + c_3).$$

To relate c_1 and c_3 to the number of holes, we use Euler's formula for a connected planar graph, which says $v - e + f = 2$, where v is the number of vertices, e is the number of edges, and f is the number of faces (including the unbounded face). (See Bondy and Murty (1976, p. 143).)

The vertices are the convex vertices of R, plus the endpoints of the chords. Thus, we have $v = \mu_1 + \mu_2 + 2(c_1 + c_2 + c_3)$.

The edges are the segments between vertices on the boundary of R, plus the chords. Each normal convex vertex is incident with two edges, each degenerate convex vertex with four edges, and every other vertex with three edges. Therefore, $2e = 2\mu_1 + 4\mu_2 + 6(c_1 + c_2 + c_3)$.

Finally, the number of faces is $f = p + h + 1$ where p is the number of rectangles in the partition, and h is the number of holes.

Using Euler's formula,

$$c_1 + c_3 = p + h - 1 - \mu_2 - c_2 \leqq p + h - 1.$$

Combining this with (∗∗∗) above yields $4p \leqq 4\theta + 2(p + h - 1)$, or $p \leqq 2\theta + h - 1$. □

The strategy of the proof of Theorem 2 is to show there is an antirectangle of size at least $(\bar{p}/2(\log \alpha + 1))$, where α is the maximum size of an antirectangle, and $\bar{p}$ is

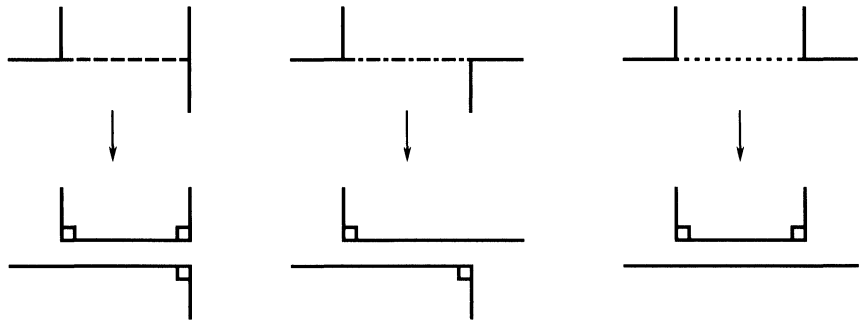

FIG. 7. *Illustration for the proof of Lemma* 2. *The top figures show the three types of chords. The bottom figures show the polygon* (*locally*) *after cutting along the chords. Each type* 1 *chord* (*left*) *yields three corners of rectangles in the partition, each type* 2 *chord* (*middle*) *or type* 3 *chord* (*right*) *yields two corners.*

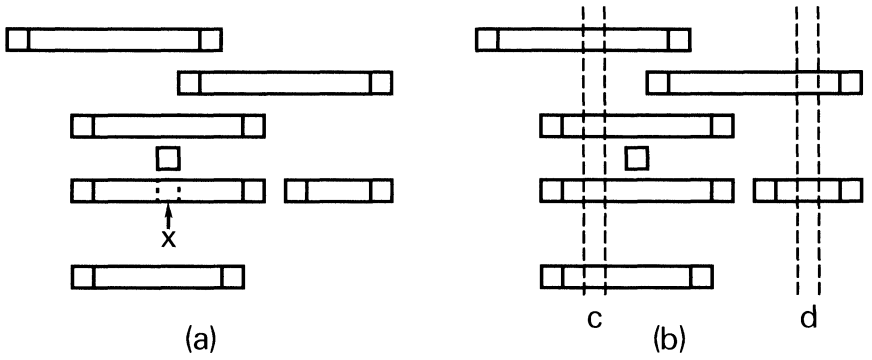

FIG. 8. *A set of inequivalent horizontal slices. The small squares are boundary squares.* (a) *Square x is blocked from above, but not from below.* (b) *Slices are separated by atoms c and d (pairs of vertical dashed lines).*

the number of rectangles in Algorithm *Partition/Extend*. The notation below will be used to describe the construction of the antirectangle.

A *horizontal slice* of R is a horizontally maximal rectangle of unit height contained in R. Call a square touching the boundary of R, but not in R, a *boundary square*. A horizontal slice is a connected row of squares in R, delimited by two boundary squares.

Each horizontal slice determines a unique maximal rectangle in R, obtained by extending the slice vertically until it meets the boundary of R on top and bottom. Two horizontal slices are *equivalent* if they determine the same maximal rectangle. A complete set of inequivalent horizontal slices determines a rectangle cover, since every square is contained in some slice. (See Figure 3(c).)

LEMMA 3. *The cover determined by a complete set of inequivalent horizontal slices is the same as the cover obtained by Algorithm Partition/Extend.*

Proof. The vertical sides of each rectangle in the horizontal partition must lie on the boundary of R. Thus, each rectangle of the partition contains a horizontal slice, so every rectangle generated by *Partition/Extend* is also generated by some slice. Conversely, every horizontal slice must be contained in some rectangle of the partition, so every rectangle generated by a slice is also generated by *Partition/Extend*. □

If x is a unit square and h is a horizontal slice, we say that x is *blocked* from h if there is no rectangle inside R containing both x and a square of h. This means that any rectangle containing x and a square of h must also contain a boundary square of R. Given a subset S of horizontal slices and a unit square x, x is *blocked from above* (with respect to S) if x is blocked from every slice in S (except the slice containing x) which is "above" x, i.e., whose vertical coordinate is greater than or equal to that of x. (See Fig. 8(a).) We define *blocked from below* similarly.

The following lemma shows that a set of blocked squares determines an antirectangle.

LEMMA 4. *Let S be a subset of horizontal slices of a rectilinear polygon R. Let A be a set of unit squares, such that each slice contains at most one square, and every square is blocked from above. Then A is an antirectangle in R.*

Proof. Let x_1 and x_2 be two squares in A, and let h_1 and h_2 be the corresponding slices. We may assume that h_2 lies above h_1 (has the same or greater vertical coordinate). Since x_1 is blocked from above, no rectangle in R contains both x_1 and x_2. □

By symmetry, the lemma also holds if "above" is replaced by "below."

Although it is not obvious how to construct a large set of blocked squares in general, if the set of slices can be "separated" by vertical lines, as described below, then we can

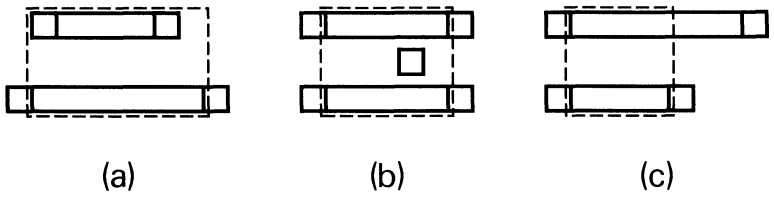

FIG. 9. *The three cases for rectangle r_i (dashed outlines), defined in Lemma 5. (The small squares are boundary squares.) I is the projection of the top slice on the x axis. J is the projection of the bottom slice.* (a) $I \subset J$. *(r_i contains a boundary square.)* (b) $I = J$. *(Since h_i and h_{i-1} are inequivalent slices, there must be a boundary square between them, in r_i.)* (c) $J \subset I$. *(r_i may not contain any boundary squares.)*

show that either half the slices contain a square blocked from above or half contain a square blocked from below.

Call an infinite vertical strip of unit width an *atom*. A slice *contains* an atom a if the slice intersects the vertical strip. Given a set of horizontal slices, S, a set of atoms $\{a_1, a_2, \cdots, a_k\}$ *separates* S if every slice contains exactly one atom. (See Fig. 8(b).) (Note that not every set of slices has such a set of atoms; see Figure 3(c).)

LEMMA 5. *Let S be a subset of inequivalent horizontal slices of a rectilinear polygon R. If atoms $\{a_1, a_2, \cdots, a_k\}$ separate S, then there is an antirectangle in R containing at least $\lceil (|S| + k)/2 \rceil$ squares.*

Remark. It is possible to construct an example with $k = 1$ where this bound is tight.

Proof. For a given atom $a \in \{a_1, a_2, \cdots, a_k\}$, let $h_1, h_2, \cdots, h_m$ be the set of slices containing a, ordered from top to bottom.

Let x_1 be the intersection of atom a and slice h_1. The square x_1 is blocked from above, since any slice h above h_1 does not contain a, so that a rectangle containing x_1 and a square of h must contain one of the boundary squares delimiting h.

For each slice h_i, $2 \leqq i \leqq m$, let r_i be the smallest rectangle which contains the slice h_i and extends to the row containing the slice h_{i-1} above. (See Fig. 9.) If r_i contains no boundary squares, then let x_i be null. Otherwise, choose a boundary square y in r_i whose horizontal distance from atom a is minimized. (Possibly, y is contained in a.) Let x_i be the square on h_i directly below y. By the definition of r_i, y lies in the same row or below that of h_{i-1}. (See Fig. 10.)

We claim that x_i is blocked from above. If not, there is a slice h, above h_i, containing a square x, such that x and x_i are in the same rectangle. This slice h must intersect the vertical strip containing y and x_i. Since y is in the same row or below h_{i-1}, h cannot contain a. Thus, one of the boundary squares at the endpoint of h lies in r_i and is closer to a than y, contradicting the choice of y. (See Fig. 10.)

For each of the k atoms, construct the corresponding set of blocked squares, $\{x_1, x_2, \cdots, x_m\}$. Let A be the union of these sets. A is a set of squares satisfying the conditions of Lemma 4, and hence is an antirectangle. Using a symmetric construction, working from bottom to top at each atom, we can construct an antirectangle B where each square is blocked from below.

The key observation is that, for any atom, if slice h_i does not contain a square in A then slice h_{i-1} must contain a square in B. To see this, let I and J be the intervals on the horizontal axis which are the projections of h_{i-1} and h_i, respectively. $I \cap J \neq \phi$ since h_{i-1} and h_i both contain a. The only case in which h_i does not contain a square of A is when r_i contains no boundary squares. Note that if $J = I$, since there are no equivalent slices in S, there must be a boundary square between h_{i-1} and h_i, in r_i. Thus, h_i contains no square of A only if J is strictly contained in I. (See Fig. 9(c).) By inspection, h_{i-1} must contain a square in B.

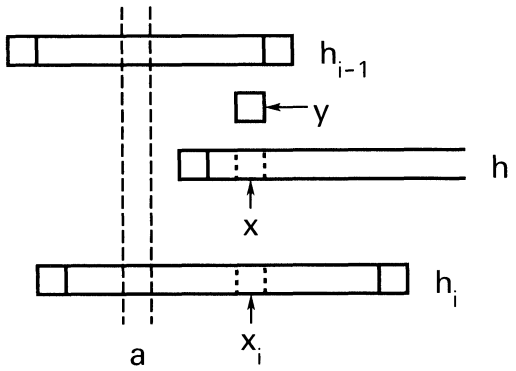

FIG. 10. *Definition of square x_i in Lemma 5, and illustration of the proof that x_i is blocked from above. Square y is the boundary square in rectangle r_i closest to atom a (vertical dashed lines). If x_i is not blocked from above, there is a slice h not containing atom a, and containing a square x, directly above x_i, as shown.*

Thus, the number of squares in B is at least the number of "bottom" slices, i.e., the number of atoms plus the number of slices which do not contain a square of A. So, $|B| \geqq k + |S| - |A|$, or $|B| + |A| \geqq |S| + k$. The lemma follows from the pigeonhole principle. □

The construction of Lemma 5 cannot be used when a slice contains more than one atom. The next two lemmas will be used to show that we can extract a large separated subset in this case.

LEMMA 6. *Let S be a set of slices, and let $\{a_1, a_2, \cdots, a_k\}$ be a set of atoms, sorted from left to right, such that each slice contains at least one atom. Then there is an m, with $m \leqq 1 + \log k$, such that S can be partitioned into disjoint sets $S = S_1 \cup S_2 \cup \cdots \cup S_m$, where each S_i is separated by a subset of the atoms.*

Proof. The proof is by induction on k. If $k = 1$ then S is separated by the single atom a, so $m = 1$.

If $k > 1$, let $j = \lceil k/2 \rceil$ and let S_1 be the set of slices containing a_j. Trivially, S_1 is separated by a_j. Observe that S is the disjoint union $S_L \cup S_1 \cup S_R$, where S_L is the set of slices to the left of a_j, and S_R is the set of slices to the right.

Since the atoms are sorted from left to right, S_L, with $\{a_1, a_2, \cdots, a_{j-1}\}$, and S_R, with $\{a_{j+1}, a_{j+2}, \cdots, a_k\}$, each satisfy the induction hypothesis. However, given a separated set in S_L and a separated set in S_R, their union is also separated. If S_L can be partitioned into m_L separated sets, and S_R can be partitioned into m_R separated sets, then

$$m \leqq 1 + \max\{m_L, m_R\}.$$

By induction, $m_L \leqq 1 + \log(j-1) \leqq \log k$, and $m_R \leqq 1 + \log(k-j) \leqq \log k$. Therefore, $m \leqq 1 + \log k$. □

If S is a set of slices, let $I(S)$ denote the set of (open) intervals obtained by projecting slices onto the horizontal axis.

LEMMA 7. *Let S be a set of slices. Let δ be the maximum number of pairwise disjoint intervals in $I(S)$. Then, there is a set of δ atoms, such that each slice contains at least one atom.*

Proof. This is a well-known duality theorem for interval graphs. The lemma says that if δ is the size of a maximum independent set in the overlap graph determined by $I(S)$, then δ is also the size of a minimum cover by cliques. See Berge (1975, p. 9) for a proof. □

THEOREM 2. *For any rectilinear polygon*, $\bar{p} \leqq 2\alpha(\log \alpha + 1)$.

Proof. Let S be the set of all inequivalent horizontal slices of R. If δ is as in Lemma 7, then there is a set of δ atoms such that each slice contains at least one atom. By Lemma 6, $S = S_1 \cup S_2 \cup \cdots \cup S_m$, where $m \leqq \log \delta + 1$, and each S_i is separated by one or more atoms.

$|S| = \bar{p}$, by Lemma 3. From the pigeonhole principle, there is some S_i with

$$|S_i| \geqq \frac{|S|}{\log \delta + 1} = \frac{\bar{p}}{\log \delta + 1}.$$

Using Lemma 5, we can construct an antirectangle from S_i, having size at least $\frac{1}{2}\bar{p}/(\log \delta + 1)$. Since $\alpha \geqq \delta$, the antirectangle has size at least $\frac{1}{2}\bar{p}/(\log \alpha + 1)$, or $\bar{p} \leqq 2\alpha(\log \alpha + 1)$. □

Remark. We can construct a set of $k = n(\log n + 1)$ intervals, with n intervals of each length in $\{1, 2, 4, 8, \cdots\}$, such that the largest subset that is separated contains $2n - 1 \in O(\log k)$ intervals. Thus, this is essentially the best bound that can be proved using Lemma 5.

4. Implementation of the heuristic. To implement *Partition/Extend*, we first make a list of all "top corners" of rectangles in the horizontal partition. (See Fig. 11(b).) From this, it is easy to extract a list of all top edges of partition rectangles. (See Fig. 11(c)). To do this, we must find the corners which are not vertices of R but are endpoints of (type 1) horizontal chords. We find the corners which are right endpoints of chords by running a vertical sweep-line from left to right across R and updating the set of chords crossing the sweep-line at each step. We then perform the mirror-image procedure to find the vertices which are left endpoints. (The use of sweep-lines is a standard method in computational geometry; see, e.g., Preparata and Shamos (1985).)

Once we have a list of top edges of rectangles in the partition, we use a similar sweep-line technique to find the upper and lower edges of the vertically extended rectangles. We sweep from bottom to top to find the upper edges, then reverse direction to find the lower edges.

The $O(n \log n)$ time bound is based on the bound for sorting, and on the use of balanced trees. Here, n is the number of vertices. If linked lists are used instead, the algorithm becomes $O(n^2)$. (Note that Ohtsuki (1982) reported the existence of an $O(n \log n)$ algorithm for horizontal partitioning but gave no details.) We give an informal presentation of further details and discussion of data structures below.

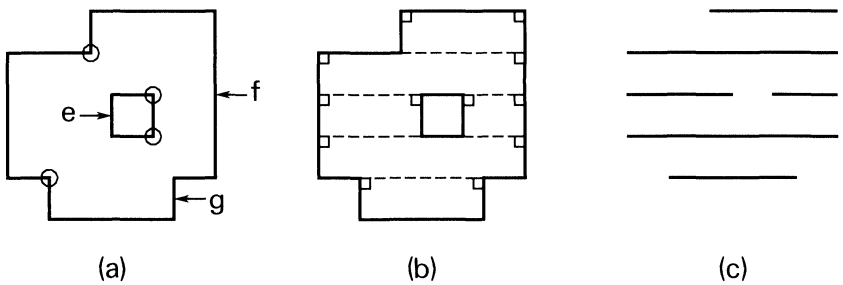

FIG. 11. (a) *Polygon with left concave corners circled, and right vertical edges labeled e, f, g.* (*Step* 1.1 *of implementation.*) (b) *Top corners of rectangles in the partition* (*marked with boxes*). *The dashed lines are horizontal chords.* (*Step* 2.) (c) *Top edges of rectangles in the partition.* (*Step* 3.)

ALGORITHM *Partition/Extend*.

Input: Rectilinear polygon R, represented as a list of vertices, sorted in order around each boundary component, oriented so that R lies to the left. (If R is connected, the outer boundary is oriented counterclockwise, and the hole boundaries are oriented clockwise.) (Note: any degenerate convex vertex will appear twice on the vertex list.)

Output: List of rectangles in a cover for R, each represented by a quadruple $[a, b, c, d]$, where (a, b) is the projection of the rectangle on the x (horizontal) axis, and (c, d) is the projection on the y (vertical) axis.

Part 1. **procedure** *Right endpoints*

{ *Left endpoints* is the mirror image of this procedure. }

Output: For each horizontal chord whose left endpoint is a concave vertex, output the right endpoint, (x, y) (the intersection of the chord with the boundary of R).

{ Outline: To find the set of right endpoints, run a vertical sweep-line from left to right, stopping at each right vertical edge. At each stopping point, update the set of chords which extend rightward from some left concave vertex, across the sweep-line. Then, find all chords which intersect a right vertical edge on the sweep line and output their endpoints. }

Step 1.1. Making one pass over the vertex list of R, extract a list V_L, of all *left concave vertices*, and a list E_R, of all *right vertical edges* of R, each represented as a triple $[x, y_1, y_2]$, where (y_1, y_2) is the projection on the y axis. (See Fig. 11(a).)

{ V_L represents the set of all chords whose left endpoint is concave. Any such chord must have its right endpoint on an edge in E_R. }

Step 1.2. Sort the edges in E_R, and the vertices in V_L from left to right.

Step 1.3. Let a be the minimum x-coordinate of the set of edges in E_R.

Let $e = [a, y_1, y_2]$, be an edge of E_R.

Delete e from E_R.

Add to L the set of vertices (x, y) in V_L, with $x < a$. Delete these vertices from V_L.

{ L represents the set of chords whose left endpoint lies strictly to the left of the line $x = a$ and whose right endpoint lies on or to the right of the line. }

Step 1.4. For each vertex (x, y) in L, with $y_1 \leqq y \leqq y_2$,

{ there must be a chord with left endpoint (x, y), which intersects edge e. }

Delete (x, y) from L;

if $y_1 < y < y_2$, then add the point (a, y) to the output list.

Step 1.5. Repeat Steps 1.3 and 1.4 until E_R or V_L is empty. □

Part 2. **procedure** *Top corners*

Output: Set of vertices which are "top corners" of rectangles in the horizontal partition. (See Fig. 11(b).)

Make one pass over the vertex list of R, examining incident edges, extracting all vertices which are either "upper convex vertices" or "lower concave vertices." To these, add the vertices found in Part 1. □

Part 3. **procedure** *Top edges*

Output: Set of top edges of rectangles in the partition, represented as triples $[x_1, x_2, y]$, where (x_1, x_2) is the projection on the x axis. (See Fig. 11(c).) Sort the list of top corners from Part 2, from top to bottom, then from left to right. Form top edges from adjacent pairs of vertices with the same y coordinate. □

Part 4. **procedure** *Upper boundaries*

{ *Lower boundaries* is the mirror image of this procedure. }

Output: List of top edges of vertically maximal rectangles represented as triples $[x_1, x_2, d]$, where (x_1, x_2) is the projection on the x axis, and d is the y-coordinate.

{ Note: when *Lower boundaries* is implemented, the input should be the list of triples $[a, b, d]$ from *Upper boundaries*. The output should be a list of quadruples $[a, b, c, d]$. }

{ Outline: use a sweep-line procedure, similar to that of *Right endpoints*, working from bottom to top, maintaining a list of all rectangles which extend upward to the current position of the sweep-line. }

Step 4.1. Let T be the list of top edges, from Part 3.

From the vertex list of R, extract a list U of all "upper horizontal edges" of R.

Step 4.2. Sort T and U from bottom to top.

Step 4.3. Let e be a horizontal edge $e = [x_1, x_2, d]$ in U with minimum y coordinate d.

Delete e from U.

Add to B all edges $[a, b, y]$ in T, with $y \leqq d$.

Delete these edges from T.

{ B contains all top edges of rectangles which can be extended upward at least to the line $y = d$. }

Step 4.4. For each edge $f = [a, b, c]$ in B, such that the (open) intervals (a, b) and (x_1, x_2) (the projection of edge e) intersect (in more than one point).

Delete f from B;

Output the edge $[a, b, d]$.

Step 4.5. Repeat Steps 4.3 and 4.4 until U or T is empty. □

Part 5. **procedure** *Delete duplicate rectangles*.

Sort the list of rectangles lexicographically, and remove duplicates. □

Let n be the number of vertices of R. Part 2 requires only $O(n)$ steps, while Parts 3 and 5 each require $O(n \log n)$ steps for sorting.

If L is stored as a linked list, then Step 1.4 may require $\Theta(|L|)$ steps, yielding an $\Theta(n^2)$ algorithm. If L is stored as a balanced tree (sorted by y coordinate), all vertices in L with $y_1 \leqq y \leqq y_2$ can be found in $O(\log |L| + k)$ steps, where k is the number of vertices found. Each addition and deletion also requires only $O(\log |L|)$ steps. (For a discussion of balanced trees, see Knuth (1973, Chap. 6.2.3).) Step 1.4 thus requires a total of $O(n \log n)$ steps. Since E_R and V_L are sorted, Step 1.3 adds only $O(n)$ steps. In either case, $O(n)$ extra space is used.

The analysis of Part 4 is the same, with one exception. Instead of using a balanced tree to store B, use a *Group Tree* (due to McCreight, described in Ullman (1984, pp. 385–391)). (This data structure was introduced independently by Edelsbrunner, who called it an *Interval Tree*, described in Preparata and Shamos (1985, pp. 352–355).) Using this data structure, we can find all intervals (a, b) which intersect the query interval (x_1, x_2) in $O(\log n + k)$ steps, where k is the number of intersections reported. Additions and deletions can also be performed in $O(\log n)$ steps each. Thus, the procedure can be performed in $O(n \log n)$ steps.

5. Concluding remarks and open problems. The main question raised by this work is whether there is any algorithm which always produces a cover with at most $c \cdot \theta$ rectangles, where c is a constant. Note that a proof which uses the technique of constructing a large antirectangle, as in the proof of Theorem 2, will prove the stronger result that θ/α is bounded.

We have shown that a simple, efficient heuristic for producing a rectangle cover yields a number of rectangles which is at worst twice the minimum if the polygon has no holes. We conjecture that the number of rectangles is within a constant factor of the minimum for an arbitrary rectilinear polygon, though we have not been able to prove this. The bound of $O(\theta \log \theta)$, from Theorem 2, is the same asymptotic bound obtained by Johnson (1974) on a heuristic for general set covering. His algorithm requires repeated searching for the maximal rectangle containing the most uncovered squares. The example of Fig. 4(a) has $\Theta(n^2)$ maximal rectangles, so a naïve implementation would use much more space and time than *Partition/Extend*.

There is some evidence that the upper bound for Algorithm *Partition/Extend* is 3θ. Observe that if there are no type 3 chords (as in Figs. 4(a) and 5), then the proof of Theorem 1 shows that $p \leqq 3\theta$. If no four vertices lie on the same horizontal line, then there can be no type 2 or type 3 chords, so $p \leqq 3\theta$ in this case also. On the other hand, the existence of many type 3 chords (or many vertices on the same line) may force rectangles to be deleted when rectangles are "extended," as in Figure 4(b).

There are many other algorithms which would be natural for further study. An obvious improvement to *Partition/Extend* is to use it both horizontally and vertically, then take the best cover. Another improvement is to find "degenerate" rectangles in the cover, those that are completely covered by a set of narrower rectangles, deleting them until stuck. (See Fig. 12.) In the example of Fig. 4(a), there is one degenerate rectangle. In Fig. 5, deleting degenerate rectangles yields an optimal cover.

Inserting or deleting one rectangle at a time, in a greedy manner, can yield covers with $\Omega(\theta^2)$ rectangles if done arbitrarily. However, there may be rules for selecting the next rectangle, with a much better bound (as in Johnson (1974)). A different type of heuristic was described informally by Hegedüs (1982). The idea is to first cover all "edge"

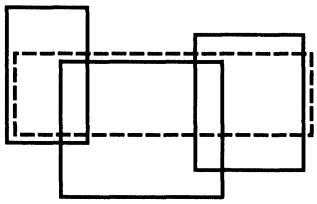

FIG. 12. "*Degenerate*" *rectangle* (*dashed outline, offset slightly*). *It is covered completely by the three rectangles with solid outlines.*

squares, greedily, obtaining a "reduced polygon," then repeating the process until all squares are covered. Note that finding a minimum cover for the set of edge squares is also NP-hard (Conn and O'Rourke (1987)).

Finally, there are several nonrectilinear covering problems that have practical applications, for which it would be worthwhile to study the performance of approximation algorithms. One problem is covering a (nonrectilinear) polygon with rectangles, allowing rotations (and assuming either no acute angles, or a means of approximating an acute angle with rectangles). This problem has been studied by Levcopoulos and Lingas (1984), and by Levcopoulos (1985). Allowing a discrete set of edge orientations, such as 45° and 90°, would also be of interest in this case. Another problem is covering an arbitrary polygon with convex polygons (proved NP-hard by O'Rourke and Supowit (1983)). There is a rather complicated polynomial-time algorithm for finding a minimum partition (when there are no holes), due to Chazelle and Dobkin (1979). It would be interesting to know whether a relation between covering and partitioning, similar to that of Theorem 1, holds in this case also.

REFERENCES

C. BERGE (1975), *Perfect graphs*, Studies in Graph Theory, Part I, D. R. Fulkerson, ed., MAA Stud. Math., 11, pp. 1–22.

J. A. BONDY AND U. S. R. MURTY (1976), *Graph Theory with Applications*, North-Holland, Amsterdam.

S. CHAIKEN, D. J. KLEITMAN, M. SAKS, AND J. SHEARER (1981), *Covering regions by rectangles*, SIAM J. Algebraic Discrete Methods, 2, pp. 394–410.

B. M. CHAZELLE AND D. DOBKIN (1979), *Decomposing a polygon into its convex parts*, In Proc. 11th ACM Sympos. on Theory of Computing, (STOC 1979), pp. 33–48.

H. E. CONN AND J. O'ROURKE (1987), *Some restricted rectangle covering problems*, Johns Hopkins University, Baltimore, MD, submitted.

J. CULBERSON AND R. A. RECKHOW (1988), *Covering polygons is hard* (preliminary abstract), Proc. 29th IEEE Sympos. Found. of Comp. Sci. (FOC 1988), pp. 601–611.

D. S. FRANZBLAU AND D. J. KLEITMAN (1984), *An algorithm for covering polygons with rectangles*, Inform. and Control, 63, pp. 164–189. Extended abstract, *An algorithm for constructing regions with rectangles*, in Proc. 16th ACM Sympos. on Theory of Computing, (STOC 1984), pp. 167–174.

M. S. GAREY AND D. S. JOHNSON (1979), *Computers and Intractability: A Guide to the Theory of* NP-*Completeness*, W. H. Freeman, San Francisco, CA.

E. GYÖRI (1984), *A minimax theorem on intervals*, J. Combin. Theory Ser. B, 37, pp. 1–9.

A. HEGEDÜS (1982), *Algorithms for covering polygons by rectangles*, Comput. Aided Geom. Design, 14, pp. 257–260.

D. S. JOHNSON (1974), *Approximation algorithms for combinatorial problems*, J. Comput. System Sci., 9, pp. 256–278.

J. M. KEIL (1986), *Minimally covering a horizontally convex orthogonal polygon*, in Proc. 2nd. ACM Symp. on Computational Geometry, pp. 43–51.

D. KNUTH (1973), *The Art of Computer Programming*, Vol. III, Addison-Wesley, Reading, MA.

C. LEVCOPOULOS (1985), *A fast heuristic for covering polygons by rectangles*, Tech. Rep. S-581 83, Linköping University, Linköping, Sweden.

C. LEVCOPOULOS AND A. LINGAS (1984), *Covering polygons with minimum number of rectangles*, in Lecture Notes in Comput. Sci. 166, Springer-Verlag, New York, Berlin, pp. 63–72.

A. LINGAS (1982), *The power of non-rectilinear holes*, 9th Int. Colloq. Autom. Lang. Prog., (ICALP 1982).

A. LUBIW (1985), Orderings and some combinatorial optimization problems with some geometric applications, Ph.D. Thesis, University of Toronto, Ontario, Canada.

——— (1988a), *The Boolean basis problem, and how to cover polygons by rectangles*, University of Waterloo, Ontario, Canada, in preparation.

——— (1988b), *A Weighted* Min Max *Relation for Intervals*, University of Waterloo, Ontario, Canada, unpublished manuscript.

W. J. MASEK (1978), *Some* NP-*complete set covering problems*, Massachusetts Institute of Technology, Cambridge, MA, unpublished manuscript. Referenced in Garey and Johnson (1979), p. 232.

C. MEAD AND L. CONWAY (1980), *Introduction to* VLSI *Systems*, Addison–Wesley, Reading, MA, Chaps. 2 and 4.

R. MOTWANI, A. RAGHUNATHAN, AND H. SARAN (1988a), *Perfect graphs and orthogonally convex covers*, University of California at Berkeley, submitted. Presented at 4th SIAM Conference on Discrete Mathematics, San Francisco, June 1988.

——— (1988b), *Covering orthogonal polygons with star polygons*: *The perfect graph approach*, in Proc. 4th ACM Symp. on Computational Geometry, pp. 211–223.

T. OHTSUKI (1982), *Minimum dissection of rectilinear regions*, Proc. 1982 IEEE Symp. on Circuits and Systems, Rome, pp. 1210–1213.

J. O'ROURKE AND K. J. SUPOWIT (1983), *Some NP-hard polygon decomposition problems*, IEEE Trans. Inform. Theory, 29, pp. 181–190.

L. PAGLI, E. LODI, F. LUCCIO, C. MUGNAI, AND W. LIPSKI (1979), *On two dimensional data organization* 2, Fund. Inform., 2, pp. 211–226.

F. P. PREPARATA AND M. I. SHAMOS (1985), *Computational Geometry*, Springer-Verlag, Berlin, New York.

R. A. RECKHOW (1987), *Covering orthogonally convex polygons with three orientations of dents*, Tech. Rep. TR 87-17, Dept. of Comp. Sci., University of Alberta, Edmonton, Canada.

R. A. RECKHOW AND J. CULBERSON (1987), *Covering a simple orthogonal polygon with a minimum number of orthogonally convex polygons*, Proc. 3rd. ACM Symp. on Computational Geometry, pp. 268–277; *Orthogonally convex coverings of orthogonal polygons without holes*, J. Comput. System Sci., to appear.

M. SAKS (1982), *A class of perfect graphs associated with planar rectilinear regions*, SIAM J. Algebraic Discrete Methods, 3, pp. 330–342.

J. D. ULLMAN (1984), *Computational Aspects of* VLSI, Computer Science Press, Rockville, MD.